Отчёт по лабораторной работе №3

Сбор данных из REST API

1. Введение

Целью работы являлся сбор структурированных данных из публичного REST API с последующей нормализацией и объединением в единый датасет. В качестве источника данных был выбран сервис DummyJSON (https://dummyjson.com) — бесплатный мок-API, предоставляющий синтетические (фейковые) данные для прототипирования и тестирования.
Были собраны данные из трёх эндпоинтов: /comments, /posts, /users, после чего произведено их объединение (merge) по связанным ключам (post_id, user_id) в единый обогащённый датасет.
В рамках работы решались следующие задачи:

- Паспортизация API и фиксация его технических характеристик.
- Обоснование выбора инструментария (Python против low-code платформ).
- Оценка правовых и этических аспектов сбора данных.

2. Паспорт API

Поле	**Значение**
Название и базовый URL	DummyJSON, https://dummyjson.com
Версия и способ её указания	Явное версионирование в URL отсутствует (нет префиксов вроде /v1/), используется текущая версия API по умолчанию.
Используемые эндпоинты	/comments (комментарии), /posts (посты), /users (пользователи, с фильтрацией полей через параметр select).
Аутентификация	Отсутствует. API полностью публичный, не требует API-ключей или Bearer-токенов.
Тип пагинации и параметры	Офсетная (offset/limit) пагинация. Основные параметры: limit (размер страницы) и skip (смещение). В коде абстрагировано через параметр page_size.

Документированные лимиты (запросов/мин)	Официально строгие лимиты не задокументированы (сервис позиционируется как бесплатный мок для тестов). На стороне клиента в коде реализован программный Rate Limiter: интервал 0.25 сек (до 240 запросов в минуту).
Заголовки лимитов в ответе	Специфичные заголовки лимитов (например, X-RateLimit-Limit) не возвращаются. В коде предусмотрен парсинг стандартного HTTP-заголовка Retry-After для обработки кодов 429 и 5xx.
Формат дат	ISO 8601 (например, YYYY-MM-DD для дат рождения или YYYY-MM-DDTHH:mm:ss.sssZ для временных меток).
Ключ уникальности записи	Поле id (целочисленный идентификатор). В коде он используется для нормализации и последующих merge (соединений) таблиц по comment_id, post_id, user_id.
Наличие персональных данных	Структурно присутствуют поля, относящиеся к ПДн (firstName, lastName, username, age, gender в эндпоинте /users), но это синтетические (фейковые) данные, сгенерированные для тестирования. Реальных ПДн нет.
Ограничения условий использования	Сервис предоставляется бесплатно (MIT / Free to use) для разработки, прототипирования и тестирования фронтенда. Запрещено использование в качестве реального продакшн-бэкенда. В коде задаётся этичный User-Agent: lab3-rest-api/1.0.

3. Обоснование выбора инструментария

Для реализации задачи рассматривались два подхода: написание скрипта на Python и использование визуальных low-code платформ (Zapier / n8n). Сравнительный анализ представлен в таблице.

Критерий	**Python**	**Zapier / n8n**
Время разработки, мин	90-120	30-60
Число собранных записей	300 (или любое другое, ограничено только аргументом max_records)	300 (ограничено настройками ноды или лимитами бесплатного тарифа)
Время выполнения одного прогона, с	~1.2 (быстрее за счет отсутствия накладных расходов облачной платформы и прямого выполнения requests)	2.836
Поддержка пагинации	Да (гибкая, полностью контролируемая кастомная функция paginate)	Да (встроенная опция "Paginate" в HTTP-ноде, но менее гибкая)
Обработка ошибок и повторов	Да (продвинутая: экспоненциальный backoff, jitter, детальное логирование в DataFrame/CSV)	Да (базовая: встроенный "Retry on Fail", но для сложной логики нужны дополнительные ноды)
Воспроизводимость (Git, экспорт)	Отличная (код хранится в .py файлах, нативная и удобная работа с Git)	Хорошая (экспорт workflow в JSON, есть интеграция с Git, но работать с диффами менее удобно)
Стоимость при 100 прогонах в месяц	0	0
Порог входа (субъективно 1–5)	4 (требуются знания синтаксиса Python, ООП, библиотек requests, pandas)	2 (интуитивный визуальный интерфейс, low-code, не требует глубоких знаний программирования)

Вывод по выбору инструмента. Несмотря на более высокий порог входа и время разработки, Python был выбран в качестве основного инструмента по следующим причинам:

1. Гибкость обработки ошибок. Реализованный механизм повторов с экспоненциальным backoff, случайным jitter и парсингом заголовка Retry-After обеспечивает надёжную работу в нестабильных сетевых условиях — функционал, который в low-code платформах потребовал бы сборки сложных ветвящихся workflow.

2. Прозрачность и воспроизводимость. Весь код хранится в текстовых файлах, что позволяет вести его версионирование через Git, рецензировать изменения и гарантировать идентичность результатов при повторных запусках.
3. Производительность. Локальный скрипт отрабатывает прогон за ~1.2 секунды, что более чем в 2 раза быстрее облачного n8n (2.836 с) за счёт отсутствия накладных расходов на маршрутизацию и визуальное исполнение.
4. Бесплатность. Решение не требует подписок и может быть запущено локально или в рамках бесплатных CI/CD.
5. Удобство последующей обработки. Использование pandas и json_normalize позволяет на том же стеке выполнить нормализацию вложенных JSON-структур и объединение таблиц через merge без экспорта в промежуточные форматы.

4. Правовая оценка сбора данных

При сборе данных из внешних API необходимо учитывать три уровня правового регулирования: условия использования сервиса, законодательство о персональных данных и этические нормы.
Сервис DummyJSON позиционируется как бесплатный мок-API с синтетическими данными, распространяемый под свободной лицензией. Согласно официальной документации, API предназначен для разработки, прототипирования и тестирования фронтенда. Использование данных в учебных и исследовательских целях прямо соответствует заявленному назначению сервиса. Ограничения касаются использования API в качестве продакшн-бэкенда, что в рамках данной работы не осуществляется.
Эндпоинт /users структурно содержит поля, которые в случае реальных данных квалифицировались бы как персональные данные (ПДн) согласно 152-ФЗ "О персональных данных" (ФИО, возраст, пол, username) и GDPR (ст. 4). Однако ключевой факт: все данные в DummyJSON являются синтетическими — они не соответствуют реальным физическим лицам и сгенерированы специально для демонстрационных целей.
Следовательно:

- Отсутствует субъект ПДн — нет реального человека, чьи данные обрабатываются.
- Не возникает правовых оснований для обработки (согласие, договор, законный интерес) — так как обрабатывать нечего.
- Нормы 152-ФЗ и GDPR в части ПДн не применяются к данному датасету.

Тем не менее, в качестве профилактической меры в коде реализована выборочная загрузка полей через параметр select=id,firstName,lastName,username,age,gender, что демонстрирует принцип минимизации данных (data minimization, ст. 5 GDPR) — запрашивается только то, что действительно необходимо для задачи.

5. Итоговые выводы

1. В работе паспортизирован API DummyJSON, зафиксированы его ключевые характеристики: способ пагинации, отсутствие аутентификации, синтетическая природа данных.
2. Обоснован выбор Python в качестве основного инструмента: несмотря на более высокий порог входа, он обеспечивает гибкость обработки ошибок, прозрачность, воспроизводимость и бесплатность.
3. Реализовано надёжное решение с клиентским rate limiting, экспоненциальным backoff, детальным логированием и нормализацией вложенных JSON-структур.
4. Правовая оценка подтверждает полную правомерность сбора: данные синтетические, ПДн отсутствуют, использование соответствует назначению сервиса, технические меры добросовестности соблюдены.

Результатом работы является готовый к использованию обогащённый датасет, полученный законным способом с применением профессиональных инженерных практик.